NVMirror: An eBPF Just-In-Time Compiler for NVMe Controller Processors

A Self-Contained Technical Reference

NVMirror Project

May 2026

Abstract

This document provides a self-contained explanation of the NVMirror project: a Just-In-Time (JIT) compiler that translates eBPF bytecode into native machine code for NVMe storage controller processors. All concepts are introduced from first principles with concrete examples. The document is structured to convey *what* the system does, *why* each design decision was made, and *how* the implementation achieves provable correctness.

Contents

1 Background: What Problem Are We Solving? **2**
- 1.1 The Storage Performance Gap . . . 2
- 1.2 The Challenge: Running User Programs on Controller Firmware . . . 2

2 Prerequisite Concepts **2**
- 2.1 What Is a Processor Instruction? . . . 2
- 2.2 What Is Assembly Language? . . . 3
- 2.3 What Is a Compiler? . . . 3
- 2.4 What Is a Just-In-Time (JIT) Compiler? . . . 3
- 2.5 What Is eBPF? . . . 3
  - 2.5.1 eBPF Instruction Encoding . . . 4
  - 2.5.2 eBPF Instruction Classes . . . 4
  - 2.5.3 Example: A Minimal eBPF Program . . . 4
- 2.6 What Is an Instruction Set Architecture (ISA)? . . . 5
- 2.7 What Is NVMe? . . . 5

3 System Architecture **5**

4 Stage 1: Decoding **6**
- 4.1 Purpose . . . 6
- 4.2 Why Strong Typing Matters . . . 6
- 4.3 Decoded Instruction Format . . . 7
- 4.4 Algorithmic Complexity . . . 7

5 Stage 2: Verification **7**
5.1 Purpose . . . 7
5.2 Control-Flow Graph Construction . . . 8
5.3 Dataflow Analysis: Register Initialization Tracking . . . 8
5.3.1 Lattice . . . 8
5.3.2 Transfer Functions . . . 8
5.3.3 Merge at Join Points . . . 9
5.3.4 Convergence . . . 9
5.4 Stack Bounds Checking . . . 9

6 Stage 3: Lowering and Register Allocation **9**
6.1 Purpose . . . 9
6.2 The Register Allocation Problem . . . 9
6.3 Our Strategy: Fixed Mapping . . . 10
6.4 Callee-Saved Registers and the Prologue/Epilogue . . . 10
6.5 The IR (Intermediate Representation) . . . 10

7 Stage 4: Code Emission **11**
7.1 Purpose . . . 11
7.2 Instruction Encoding: AArch64 Example . . . 11
7.3 Handling Immediate Values . . . 12
7.4 Branch Handling and Relocations . . . 12
7.5 RISC-V Encoding . . . 12
7.6 The Two-Pass Strategy (Dry-Run) . . . 12

8 Stage 5: Execution **13**
8.1 The Executable Buffer . . . 13
8.2 W^X (Write XOR Execute) Policy . . . 13
8.3 On Firmware (no_std) . . . 13

9 Correctness Guarantees **14**
9.1 Correct-by-Construction Design . . . 14
9.2 Differential Testing (Translation Validation) . . . 14
9.3 The Fuzzer . . . 14

10 Concrete Walkthrough: Fibonacci **14**
10.1 Source Program (eBPF Assembly) . . . 15
10.2 After Decoding (Stage 1) . . . 15
10.3 After Verification (Stage 2) . . . 15
10.4 After Lowering (Stage 3) . . . 15
10.5 After Emission (Stage 4) . . . 16
10.6 Execution (Stage 5) . . . 16

11 Project Structure **16**

12 Testing and Validation **17**
12.1 Test Tiers . . . 17
12.2 Running . . . 17

13 Key Design Decisions and Trade-offs **17**
13.1 Why eBPF R0 Maps to X7 (Not X0) 17
13.2 Why Conservative Atomics (DMB ISH / FENCE) 17
13.3 Why No Loops in eBPF 18

14 Future Work **18**

15 Glossary **18**

1 Background: What Problem Are We Solving?

1.1 The Storage Performance Gap

Modern NVMe solid-state drives (SSDs) can deliver over 7 GB/s of bandwidth and millions of I/O operations per second. However, to *process* the data read from storage, it must travel across a PCIe bus to the host CPU, be processed, and potentially travel back. This round-trip creates latency and consumes host CPU cycles and memory bandwidth.

Key Insight

Computational storage moves computation *into* the storage device itself. Instead of transferring data to the CPU for processing, we send a small program to the drive's internal processor and let it process data in-place, near the flash media.

1.2 The Challenge: Running User Programs on Controller Firmware

NVMe controllers contain small embedded processors (typically ARM Cortex-R or RISC-V cores). These processors run firmware—fixed software burned into the device. We want to allow users to send *custom programs* that execute on this firmware, but with critical constraints:

(1) **Safety**: A user program must not crash the controller, corrupt data, or access unauthorized memory.

(2) **Performance**: Programs must run at near-native speed, not through a slow software interpreter.

(3) **Resource limits**: The controller has limited RAM (often <1 MB), no virtual memory, and no operating system.

This is where eBPF and JIT compilation enter the picture.

2 Prerequisite Concepts

2.1 What Is a Processor Instruction?

At the lowest level, a processor executes *instructions*—fixed-width binary patterns that encode operations. For example, on an ARM processor, the 32-bit pattern:

```
0xD2800547
```

means "place the number 42 into register X7." Every program you have ever run—from a web browser to a database—ultimately reduces to a sequence of such binary instructions.

Definition

A **register** is a tiny, ultra-fast storage cell inside the processor. A typical processor has 16–32 registers, each holding one 64-bit integer. All arithmetic, logic, and data movement operations work on registers.

2.2 What Is Assembly Language?

Assembly language is a human-readable notation for binary instructions. Instead of writing `0xD2800547`, we write:

```
MOVZ X7, #42      // Load the immediate value 42 into register X7
```

There is a one-to-one correspondence between assembly mnemonics and binary machine code. An *assembler* translates the text form to binary; a *disassembler* does the reverse.

2.3 What Is a Compiler?

A compiler translates a program from one language (the *source*) to another (the *target*). Typically:

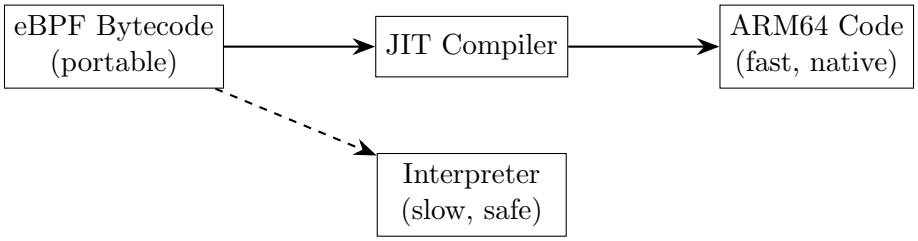
- A C compiler translates C source code into machine code for x86 or ARM.
- The translation happens *before* the program runs (**ahead-of-time** compilation).

2.4 What Is a Just-In-Time (JIT) Compiler?

Definition

A **Just-In-Time compiler** translates a program into machine code *at runtime*, immediately before executing it. The source is typically a portable *bytecode* format (not human-readable source code).

The key advantage: the input bytecode is portable (runs on any target), but execution speed equals native machine code because we compile it to the specific processor's instruction set just before running it.



2.5 What Is eBPF?

Definition

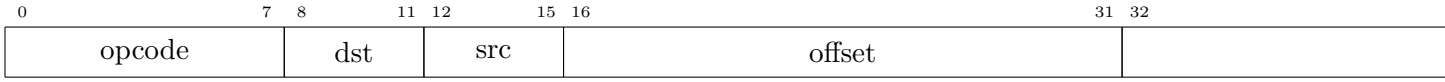
eBPF (extended Berkeley Packet Filter) is a virtual instruction set architecture (ISA) originally designed for safely filtering network packets inside the Linux kernel. It defines a simple, restricted virtual machine with:

- 11 registers (R0–R10), each 64 bits wide
- A 512-byte stack
- No loops (programs must terminate)
- No arbitrary memory access (all pointers are bounds-checked)

eBPF was designed to be *statically verifiable*—a program's safety can be proven *before* execution, without running it. This is what makes it ideal for untrusted code on embedded controllers.

2.5.1 eBPF Instruction Encoding

Each eBPF instruction is exactly 8 bytes (64 bits) with this layout:



- **opcode** (8 bits): Encodes the operation class and specific operation.
- **dst** (4 bits): Destination register number (0–10).
- **src** (4 bits): Source register number (0–10).
- **offset** (16 bits, signed): Memory offset for load/store, or branch displacement.
- **immediate** (32 bits, signed): Constant operand.

2.5.2 eBPF Instruction Classes

The low 3 bits of the opcode determine the instruction *class*:

Class	Bits [2:0]	Description
ALU	`0x04`	32-bit arithmetic/logic
ALU64	`0x07`	64-bit arithmetic/logic
JMP	`0x05`	64-bit conditional/unconditional jumps
JMP32	`0x06`	32-bit conditional jumps
LD	`0x00`	Load from packet/immediate
LDX	`0x01`	Load from memory
ST	`0x02`	Store immediate to memory
STX	`0x03`	Store register to memory

2.5.3 Example: A Minimal eBPF Program

Consider a program that returns the number 42:

```
MOV64 R0, 42     // R0 = 42 (return value register)
EXIT             // End program, return R0 to caller
```

Encoded as raw bytes (two 8-byte instructions):

```
B7 00 00 00 2A 00 00 00     // opcode=0xB7 (ALU64|MOV|K), dst=R0, imm=42
95 00 00 00 00 00 00 00     // opcode=0x95 (JMP|EXIT)
```

2.6 What Is an Instruction Set Architecture (ISA)?

Definition

An **ISA** is the contract between software and hardware. It specifies:

- What instructions exist (ADD, MUL, LOAD, BRANCH, etc.)
- How many registers are available and how wide they are
- How instructions are encoded in binary
- How memory is addressed

NVMirror targets two ISAs:

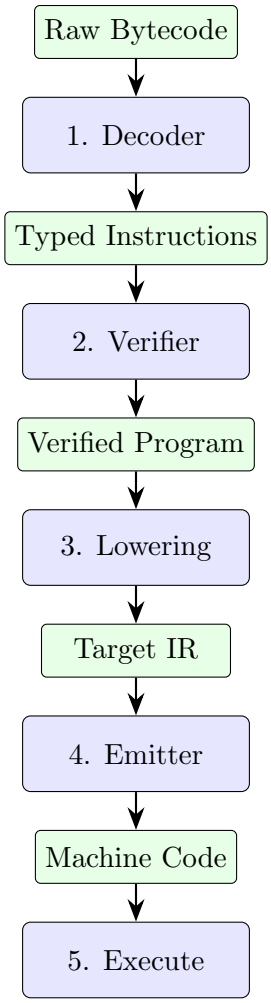
- **AArch64** (ARM 64-bit): Used in Apple M-series chips, NVIDIA Jetson, and ARM Cortex-R82 NVMe controllers. Instructions are fixed at 32 bits wide.
- **RISC-V RV64**: An open-source ISA used in emerging NVMe controllers. Base instructions are 32 bits wide.

2.7 What Is NVMe?

NVMe (Non-Volatile Memory Express) is the standard interface protocol for SSDs connected via PCIe. An NVMe controller is the embedded system inside the SSD that manages flash memory, handles I/O commands, and now (with NVMirror) can execute computational programs.

3 System Architecture

NVMirror is structured as a **multi-stage pipeline**. Each stage transforms the program into a representation closer to executable machine code, while maintaining safety invariants.



4 Stage 1: Decoding

4.1 Purpose

The decoder (`crates/ebpf-core/src/decode.rs`) transforms raw 8-byte instruction encodings into a *strongly typed* Rust data structure. This is analogous to parsing—converting an unstructured byte stream into a structured abstract syntax tree.

4.2 Why Strong Typing Matters

In the raw encoding, a register number is just 4 bits that could contain any value from 0 to 15. But eBPF only has registers 0–10. By parsing into a `BpfReg` type that *rejects* values above 10 at construction time, we make invalid register references **unrepresentable** in our program. This is a form of *correct-by-construction* design:

```
pub struct BpfReg(u8);  // Invariant: self.0 <= 10

impl BpfReg {
    pub const fn from_raw(raw: u8) -> Option<Self> {
        if raw <= 10 { Some(Self(raw)) } else { None }
    }
```

```
}
```

After decoding, every subsequent stage can assume all register references are valid without re-checking.

4.3 Decoded Instruction Format

The decoder produces a Rust `enum` (sum type) where each variant represents exactly one kind of eBPF instruction with precisely the operands it needs:

```
enum Insn {
    Alu64 { op: AluOp, dst: BpfReg, src: Source },
    Load  { width: MemWidth, dst: BpfReg, src: BpfReg, off: i16 },
    JmpCond { op: JmpOp, dst: BpfReg, src: Source, off: i16 },
    Exit,
    // ... other variants
}
```

4.4 Algorithmic Complexity

The decoder performs a single linear scan over the instruction array:

$$T(\text{decode}) = O(n) \quad \text{where } n = \text{number of instructions}$$

The only special case is the 64-bit immediate load (LDDW), which consumes two consecutive 8-byte slots to encode a full 64-bit constant.

5 Stage 2: Verification

5.1 Purpose

The verifier (`crates/ebpf-core/src/verify.rs`) is the **safety critical** component. It statically proves that the program cannot:

1. Read an uninitialized register (undefined behavior)
2. Write to R10 (the read-only frame pointer)
3. Branch outside the program's instruction array
4. Access stack memory outside the 512-byte frame
5. Fail to terminate (infinite loops)
6. Contain dead/unreachable code (potential confusion)

Important

If the verifier accepts a program, all subsequent stages can *assume* these properties hold without rechecking. A bug in the verifier compromises the entire system's safety. This is why it is the most heavily tested component.

5.2 Control-Flow Graph Construction

The verifier first builds a **Control-Flow Graph (CFG)**—a directed graph where:

- **Nodes** are basic blocks (maximal sequences of instructions with no internal branches).
- **Edges** represent possible flow: fall-through to the next block, or a branch to a target block.

Definition

A **basic block** is a sequence of instructions where:

1. The only entry point is the first instruction (no jumps into the middle).
2. The only exit point is the last instruction (which is either a branch, return, or falls through to the next block).

New basic blocks start at:

- The program entry point (instruction 0)
- Any instruction that is the target of a branch
- Any instruction immediately following a branch or exit

5.3 Dataflow Analysis: Register Initialization Tracking

The verifier performs a **fixed-point dataflow analysis** to track which registers are initialized at each point in the program.

5.3.1 Lattice

Each register has a state from the lattice {Uninit, Init}. At the program entry:

- R1 = Init (context pointer, provided by the caller)
- R10 = Init (frame pointer, always valid)
- R0, R2–R9 = Uninit

5.3.2 Transfer Functions

For each instruction, we update the register state:

- `MOV R3, 42`: Sets R3 → Init
- `ADD R0, R1`: Reads R0 and R1 (must both be Init), result in R0 remains Init
- `CALL helper`: R0 → Init (return value), R1–R5 → Uninit (clobbered)

5.3.3 Merge at Join Points

When two control-flow paths converge (e.g., after an if/else), register states are merged conservatively:

$$\text{merge}(s_1, s_2) = \begin{cases} \textsc{Init} & \text{if } s_1 = \textsc{Init} \wedge s_2 = \textsc{Init} \\ \textsc{Uninit} & \text{otherwise} \end{cases}$$

This means: a register is considered initialized at a merge point *only if* it is initialized on *all* incoming paths.

5.3.4 Convergence

The analysis iterates until no register states change (fixed point). Since the lattice has only two elements and states can only go from $\textsc{Init} \to \textsc{Uninit}$ (never the reverse) at merge points, convergence is guaranteed in at most $O(k)$ iterations where k is bounded by the number of back-edges (loops). eBPF programs have bounded loops, so:

$$T(\text{verify}) = O(n \cdot k), \quad k \leq 32$$

5.4 Stack Bounds Checking

When an instruction accesses memory relative to R10 (the frame pointer), the verifier checks:

$$-512 \leq \text{offset} < 0$$

The eBPF stack grows downward from R10. Any access at offset ≥ 0 or < -512 is rejected.

6 Stage 3: Lowering and Register Allocation

6.1 Purpose

Lowering (`crates/jit-regalloc/src/lib.rs`) transforms verified eBPF instructions into a **target-independent intermediate representation (IR)** that uses *physical registers* instead of virtual eBPF registers.

6.2 The Register Allocation Problem

eBPF programs use virtual registers R0–R10. The target processor has *physical* registers (X0–X30 on AArch64, x0–x31 on RISC-V). We must decide which physical register holds each virtual register's value.

Definition

Register allocation is the process of assigning virtual registers to physical registers. When there are more virtual registers than physical registers, some values must be *spilled* to memory (the stack) and later *filled* (reloaded).

6.3 Our Strategy: Fixed Mapping

Since eBPF has only 11 registers and both target ISAs have >30, we use a **fixed, predetermined mapping**—no complex allocation algorithm needed:

eBPF Register	AArch64	RISC-V
R0 (return value)	X7	a0
R1 (argument 1)	X0	a1
R2 (argument 2)	X1	a2
R3 (argument 3)	X2	a3
R4 (argument 4)	X3	a4
R5 (argument 5)	X4	a5
R6 (callee-saved)	X19	s2
R7 (callee-saved)	X20	s3
R8 (callee-saved)	X21	s4
R9 (callee-saved)	X22	s5
R10 (frame pointer)	X25	s6
Scratch/temp	X9	s7

Key Insight

The AArch64 mapping places eBPF R1–R5 in X0–X4. This is deliberate: when our JIT'd code calls a helper function, the arguments are already in the correct registers per the ARM calling convention, avoiding any register shuffling. This zero-cost ABI alignment is a significant performance optimization.

6.4 Callee-Saved Registers and the Prologue/Epilogue

Definition

In a **calling convention**, some registers are designated **callee-saved**: if a function modifies them, it must restore their original values before returning. Other registers are **caller-saved**: the caller assumes they may be destroyed by any function call.

On AArch64, X19–X28 are callee-saved. Since eBPF R6–R9 map to X19–X22, any program that writes to R6–R9 must:

1. **Prologue**: Save the original X19–X22 values to the stack at function entry.
2. **Epilogue**: Restore them from the stack before returning.

The lowering pass detects whether any instruction writes to R6–R9 and, if so, emits Prologue/Epilogue IR nodes.

6.5 The IR (Intermediate Representation)

The IR (`crates/jit-ir/src/lib.rs`) uses the same instruction *kinds* as eBPF but with physical registers and explicit basic-block labels:

```
enum IrNode {
    Alu64 { op: AluOp, dst: PhysReg, src: Operand },
    Load  { width: MemWidth, dst: PhysReg, base: PhysReg, off: i16 },
    Branch { cond: BranchCond, target: BasicBlockId },
    Jump { target: BasicBlockId },
    Prologue { frame_size: u16 },
    Epilogue { frame_size: u16 },
    Ret,
    // ...
}
```

Branch targets are `BasicBlockId` values (indices into an array), not raw byte offsets. This makes out-of-bounds jumps *structurally impossible*—you cannot construct an `IrNode::Branch` pointing to a nonexistent block.

7 Stage 4: Code Emission

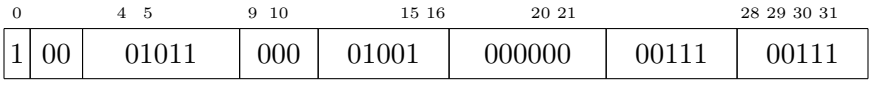
7.1 Purpose

The emitter translates each IR node into one or more *binary-encoded* target instructions. NVMirror has two emitters:

- `crates/jit-aarch64/`: Emits ARM AArch64 machine code
- `crates/jit-riscv/`: Emits RISC-V RV64IMC machine code

7.2 Instruction Encoding: AArch64 Example

On AArch64, every instruction is exactly 32 bits (4 bytes). The encoding packs the operation and operands into specific bit fields. For example, `ADD X7, X7, X9`:



- Bit 31 (sf=1): 64-bit operation
- Bits 30–29 (op=00): ADD (not SUB)
- Bits 28–24: Fixed encoding for "data processing, register"
- Bits 20–16 (Rm=01001): Second source register X9
- Bits 9–5 (Rn=00111): First source register X7
- Bits 4–0 (Rd=00111): Destination register X7

The emitter functions encode this as pure arithmetic:

```
pub const fn add_x_reg(rd: u8, rn: u8, rm: u8) -> u32 {
    0x8B000000 | ((rm as u32) << 16) | ((rn as u32) << 5) | (rd as u32)
}
```

7.3 Handling Immediate Values

eBPF instructions can have 64-bit immediates, but AArch64 instructions only have room for 12–16 bits of immediate data. To load a full 64-bit constant, we must use a multi-instruction sequence:

```
MOVZ X7, #0x5678            // X7 = 0x0000_0000_0000_5678
MOVK X7, #0x1234, LSL #16   // X7 = 0x0000_0000_1234_5678
MOVK X7, #0xABCD, LSL #32   // X7 = 0x0000_ABCD_1234_5678
MOVK X7, #0x9999, LSL #48   // X7 = 0x9999_ABCD_1234_5678
```

`MOVZ` loads a 16-bit value and zeros the rest. Each subsequent `MOVK` "keeps" the existing bits and inserts 16 new bits at the specified position. The emitter optimizes this by omitting `MOVK` for zero chunks.

7.4 Branch Handling and Relocations

Branches pose a challenge: when emitting a branch instruction, we may not yet know the *byte offset* of the target basic block (it hasn't been emitted yet). The solution:

1. Emit the branch with a **placeholder** offset of zero.
2. Record a **relocation**: "at byte offset X in the buffer, patch in the offset to basic block B."
3. After all blocks are emitted, **finalize**: walk the relocation table and patch each branch with the now-known target offset.

```
struct Relocation {
    offset: usize,         // Where in the buffer the branch lives
    target: BasicBlockId, // Which BB it should jump to
    is_conditional: bool, // B.cond (19-bit range) vs B (26-bit range)
}
```

7.5 RISC-V Encoding

RISC-V uses multiple encoding formats (R-type, I-type, S-type, B-type, U-type, J-type), each packing operands differently:

Format	Used For
R-type	Register-register arithmetic (`ADD x1, x2, x3`)
I-type	Immediate arithmetic, loads (`ADDI x1, x2, 42`)
S-type	Stores (`SD x1, 8(x2)`)
B-type	Conditional branches (`BEQ x1, x2, offset`)
U-type	Upper-immediate (`LUI x1, 0x12345`)
J-type	Unconditional jumps (`JAL x1, offset`)

7.6 The Two-Pass Strategy (Dry-Run)

In a memory-constrained firmware environment, we cannot reallocate buffers. NVMirror uses a **two-pass** approach:

1. **Dry-run pass**: Walk the IR and count the maximum number of target instructions per node (bounded constant per instruction type). Compute the exact buffer size needed.
2. **Emission pass**: Allocate exactly that many bytes, then emit for real.

The worst-case expansion ratio is a compile-time constant: each eBPF instruction maps to at most 8 AArch64 instructions (for complex cases like MOD with a large immediate).

8 Stage 5: Execution

8.1 The Executable Buffer

After emission, we have a `Vec<u8>` containing machine code. To execute it, we need to:

1. Place it in memory marked as **executable**.
2. Obtain a **function pointer** to the start of that memory.
3. Call it like a normal function.

8.2 W^X (Write XOR Execute) Policy

Modern systems enforce that memory is either writable *or* executable, never both simultaneously. This prevents code-injection attacks. The flow is:

1. Allocate memory with `mmap(MAP_JIT)` (macOS) or `mmap(PROT_READ|PROT_WRITE|PROT_EXEC)` (Linux).
2. Switch to **writable** mode: `pthread_jit_write_protect_np(0)` (macOS).
3. Copy machine code bytes into the buffer.
4. Flush the instruction cache: `sys_icache_invalidate()` (macOS) or `__clear_cache()` (Linux).
5. Switch to **executable** mode: `pthread_jit_write_protect_np(1)` (macOS).
6. Cast the buffer pointer to a function pointer and call it.

Key Insight

The instruction cache flush is critical on ARM processors. Unlike x86, ARM has a *split cache* architecture: the instruction cache (I-cache) and data cache (D-cache) are separate and not automatically coherent. After writing code as *data*, we must explicitly tell the processor to refresh the I-cache.

8.3 On Firmware (no_std)

On an actual NVMe controller without an OS, the executable buffer is a static array in SRAM. The MPU (Memory Protection Unit) is configured to make that region executable. There is no `mmap`, no page tables—just direct physical addressing.

9 Correctness Guarantees

9.1 Correct-by-Construction Design

NVMirror's primary correctness strategy is **making invalid states unrepresentable**:

Invariant	Enforcement Mechanism
Register $\in [0, 10]$	`BpfReg` newtype rejects > 10 at parse time
Branch target $\in$ program	`BasicBlockId` is an index into a verified array
Stack $\in [-512, 0)$	Verifier rejects out-of-range offsets
No uninit reads	Dataflow analysis proves initialization
Program terminates	Back-edge budget (max 32 loop iterations)

9.2 Differential Testing (Translation Validation)

NVMirror includes a **reference interpreter** that executes eBPF instructions directly in software. For every program:

$$\text{Result}_{\text{interpreter}}(\mathcal{P}) = \text{Result}_{\text{JIT}}(\mathcal{P})$$

This is checked by:

- **Conformance tests**: 32 hand-crafted programs covering all instruction classes.
- **Differential fuzzer**: Generates 1000+ random valid eBPF programs and checks that JIT output matches interpreter output.

Theorem 1 (Soundness of Differential Testing)**.** *If the reference interpreter correctly implements the eBPF specification, and for all programs $\mathcal{P}$ that pass the verifier:*

$$\forall \mathcal{P} : \mathit{verify}(\mathcal{P}) = \mathit{OK} \implies \mathit{interp}(\mathcal{P}) = \mathit{exec}(\mathit{jit}(\mathcal{P}))$$

then the JIT is a correct implementation of the eBPF semantics.

9.3 The Fuzzer

The fuzzer (`fuzz/src/lib.rs`) uses **grammar-based generation**:

1. Generate random programs that *structurally* satisfy verifier constraints (initialized registers, forward-only branches, proper EXIT).
2. Attempt compilation—some may still fail verification (that's acceptable).
3. For programs that compile, execute both via interpreter and via native JIT.
4. Assert results match.

This is more effective than byte-level mutation because it explores the *semantic* space of valid programs rather than wasting time on obviously invalid inputs.

10 Concrete Walkthrough: Fibonacci

Let us trace the compilation of a Fibonacci program through all stages.

10.1 Source Program (eBPF Assembly)

Compute fib(7) = 13 iteratively:

```
MOV64 R6, 0          // a = 0
MOV64 R7, 1          // b = 1
// Iteration 1:
MOV64 R8, R7         // tmp = b
ADD64 R7, R6         // b = b + a
MOV64 R6, R8         // a = tmp
// ... (repeat 5 more times) ...
MOV64 R0, R7         // return b
EXIT
```

10.2 After Decoding (Stage 1)

Each line becomes a typed `Insn`:

```
Alu64 { op: Mov, dst: R6, src: Imm(0) }
Alu64 { op: Mov, dst: R7, src: Imm(1) }
Alu64 { op: Mov, dst: R8, src: Reg(R7) }
Alu64 { op: Add, dst: R7, src: Reg(R6) }
Alu64 { op: Mov, dst: R6, src: Reg(R8) }
// ...
```

10.3 After Verification (Stage 2)

The verifier confirms:

- R6, R7, R8 are initialized (by MOV) before any reads.
- R0 is set (from R7) before EXIT.
- No branches ⇒ single basic block.
- No stack access ⇒ no bounds-check needed.
- Program has an EXIT ⇒ terminates.

10.4 After Lowering (Stage 3)

Virtual registers → physical registers; prologue/epilogue added because R6–R8 map to callee-saved X19–X21:

```
Prologue { frame_size: 64 }    // Save X19-X22, X25, LR, FP
Alu64 { Mov, X19, Imm(0) }     // R6 -> X19
Alu64 { Mov, X20, Imm(1) }     // R7 -> X20
Alu64 { Mov, X21, Reg(X20) }   // R8 -> X21
Alu64 { Add, X20, Reg(X19) }   // R7 += R6
Alu64 { Mov, X19, Reg(X21) }   // R6 = R8
// ... (18 more iterations) ...
Alu64 { Mov, X7, Reg(X20) }    // R0 = R7 (X7 is our R0)
Epilogue { frame_size: 64 }    // Restore callee-saved regs
Ret
```

10.5 After Emission (Stage 4)

Each IR node becomes 1–5 AArch64 instructions:

```
// Prologue (5 instructions)
STP X29, X30, [SP, #-64]!    // Save frame pointer and link register
MOV X29, SP                   // Set up new frame pointer
STP X19, X20, [SP, #16]      // Save callee-saved pair 1
STP X21, X22, [SP, #32]      // Save callee-saved pair 2
STR X25, [SP, #48]           // Save eBPF frame pointer

// Body (22 instructions: MOV, ADD sequences)
MOVZ X19, #0                 // R6 = 0
MOVZ X20, #1                 // R7 = 1
MOV X21, X20                 // R8 = R7 (tmp = b)
ADD X20, X20, X19            // R7 = R7 + R6 (b = b + a)
MOV X19, X21                 // R6 = R8 (a = tmp)
// ... repeat ...

// Epilogue + Return (5 instructions)
LDP X19, X20, [SP, #16]      // Restore callee-saved
LDP X21, X22, [SP, #32]
LDR X25, [SP, #48]
LDP X29, X30, [SP], #64      // Restore FP/LR, deallocate frame
MOV X0, X7                   // ABI: return value in X0
RET                           // Return to caller
```

10.6 Execution (Stage 5)

The 32 instructions (128 bytes) are written to a MAP_JIT buffer, the I-cache is flushed, and the code is called. Result: **R0 = 13** (= fib(7)).

11 Project Structure

Crate	Responsibility
`ebpf-core`	ISA constants, bytecode decoder, static verifier
`jit-ir`	Intermediate representation types, `CodeEmitter` trait
`jit-regalloc`	Register mapping tables, lowering from `Insn` to `IrNode`
`jit-aarch64`	AArch64 instruction encoding (`encode.rs`) and emitter
`jit-riscv`	RISC-V RV64 instruction encoding and emitter
`jit-engine`	Pipeline orchestrator, reference interpreter
`nvme-shim`	Executable buffer (mmap/MAP_JIT for userspace, static array for firmware)
`nvmirror-fuzz`	Differential fuzzer with grammar-based program generation
`nvmirror-disasm`	Pipeline visualizer (shows all 6 stages for example programs)

All crates compile with `#![no_std]` for firmware portability. The `std` feature gates userspace-only functionality (mmap, printing).

12 Testing and Validation

12.1 Test Tiers

1. **Unit tests** (per-crate): Verify individual instruction encoding, decoder edge cases, verifier rejection behavior.
2. **Integration tests**: End-to-end compilation from raw bytecode to machine code.
3. **Native execution tests**: JIT-compile and *run* on Apple Silicon, compare against interpreter.
4. **Conformance suite**: 32 test vectors covering every eBPF instruction class.
5. **Differential fuzzer**: 1000+ random programs per run.

12.2 Running

```
cargo test                                    # All 75 tests
cargo run -p nvmirror-disasm                  # Visualize pipeline
cargo run -p nvmirror-disasm -- --example fibonacci  # Single example
```

13 Key Design Decisions and Trade-offs

13.1 Why eBPF R0 Maps to X7 (Not X0)

On AArch64, X0 is both the first function argument *and* the return value register. eBPF R1 is the first argument and R0 is the return value—these are *different* registers in eBPF but the *same* register (X0) in the ARM ABI.

By mapping eBPF R1→X0 (argument passthrough) and eBPF R0→X7, we avoid a destructive conflict: writing to R0 (the return value) would clobber R1 (the first argument) if they shared X0. The cost is one extra `MOV X0, X7` before every RET.

13.2 Why Conservative Atomics (DMB ISH / FENCE)

eBPF's atomic operations (XADD, CMPXCHG, etc.) have loose memory-ordering semantics in the spec. Rather than risk subtle concurrency bugs from an under-specified ordering, we emit the strongest possible barrier:

- AArch64: `DMB ISH` (Data Memory Barrier, Inner Shareable)
- RISC-V: `FENCE RW, RW`

This is sequentially consistent—the slowest but most correct option. Once correctness is established and profiling data is available, barriers can be relaxed.

13.3 Why No Loops in eBPF

eBPF guarantees termination by requiring all branches to be *forward-only* or to *bounded loops* (verified by a back-edge budget). This eliminates the halting problem for the verifier: we can always statically determine if a program will terminate.

NVMirror enforces this with a budget of 32 back-edge traversals during dataflow analysis. Programs exceeding this are rejected.

14 Future Work

- **DMA offload detection**: Pattern-match sequential load/store loops in the IR and replace them with DMA descriptor chains (hardware-accelerated bulk transfers).
- **Peephole optimization**: Eliminate redundant MOV instructions, fuse CMP+BRANCH sequences.
- **RISC-V native execution**: Add QEMU-user-mode test harness for validating RISC-V output without hardware.
- **Formal verification**: Encode the register mapping correctness in a proof assistant (e.g., Lean4 or Coq).
- **BTF integration**: Support eBPF Type Format for rich type information in computational storage programs.

15 Glossary

ABI (Application Binary Interface)
The convention specifying how functions pass arguments and return values in registers.

Basic Block
A sequence of instructions with one entry and one exit point.

Callee-Saved Register
A register that a called function must preserve (save on entry, restore on exit).

CFG (Control-Flow Graph)
A directed graph of basic blocks connected by branch edges.

Dataflow Analysis
A technique for computing facts about program state (e.g., which registers are initialized) by propagating information along CFG edges.

DMB (Data Memory Barrier)
An ARM instruction that ensures all memory accesses before it are visible to all other observers before any memory accesses after it.

eBPF
Extended Berkeley Packet Filter—a restricted virtual ISA designed for safe, verifiable programs.

Fixed-Point
The state where repeated application of an analysis produces no further changes.

I-Cache
Instruction cache—a hardware cache holding recently fetched instructions, separate from the data cache on ARM.

IR (Intermediate Representation)
An internal data structure used by compilers between the source and target languages.

ISA (Instruction Set Architecture)
The specification of a processor's instruction repertoire.

JIT (Just-In-Time)
Compilation at runtime, immediately before execution.

MAP_JIT
A macOS `mmap` flag that allocates memory suitable for JIT compilation with W^X enforcement.

MPU (Memory Protection Unit)
Hardware in embedded processors that enforces access permissions on memory regions.

Relocation
A record indicating where in the output buffer a branch offset must be patched after all code is emitted.

Spill/Fill
Saving a register to the stack (spill) and later reloading it (fill) when physical registers are insufficient.

W^X
Write XOR Execute—a security policy where memory pages cannot be both writable and executable simultaneously.